Recitation 5 Solutions

In this problem, use **tidyverse functions only** (you are allowed to use any helper function such as mean(), median(), as.factor(), as.numeric(), and so on. Tidyverse package extensions can be used as well).

`transaction_data_1.csv` and `transaction_data_2.csv` datasets represent customer transactions at an online retail store specializing in electronics, clothing, and household items. The store collects transaction data to analyze customer behavior, improve recommendations, and optimize inventory. Each transaction includes demographic details, order details, and behavioral data such as payment method and discount usage. The goal is to use this data to personalize marketing efforts and enhance user experience.

The datasets include 800 purchases made over a three-month period and include the following variables:

transaction_data_1

- `Transaction_ID` - Unique identifier for each transaction.
- `Customer_ID` - Unique identifier for each customer.
- `Age` - Age of the customer.
- `Income` - Annual income of the customer in USD.
- `Discount_Applied` - Whether a discount was applied (Yes/No).
- `Customer_Rating` - Customer rating of the product (1-5 scale).
- `Quantity.Product_Price` - Quantity of units purchased followed by the product price (price per unit).

transaction_data_2

- `Transaction_ID` - Unique identifier for each transaction.
- `Gender` - Gender of the customer (Male, Female, Other).
- `Category` - Product category (Electronics, Clothing, Household).
- `Payment_Method` - Payment method used (Credit Card, PayPal, Debit Card, Other).
- `Shipping_Cost` - Cost of shipping in USD.
- `Delivery_Time` - Number of days taken for delivery.

```
library(tidyverse)
```

Question 1:

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
```

```
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ---------------------------------------- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(patchwork)
```

```
## Warning: package 'patchwork' was built under R version 4.4.3
```

```
library(ggExtra)

transaction_data_1 <- read.csv("transaction_data_1.csv", header = T,

                               stringsAsFactors = T)

transaction_data_2 <- read.csv("transaction_data_2.csv",header  = T,

                               stringsAsFactors = T)
```

- Clean the `transaction_data_1` and `transaction_data_2` datasets and make them tidy, that is, identify and fix all the issues introduced into the datasets.

```
transaction_data_1 <- transaction_data_1 %>%

  separate(Quantity.Product_Price, into = c("Quantity", "Product_Price"),

           convert = TRUE) %>%

  separate(Income, into = c("temp", "Income"), sep = "\\$", remove = TRUE,

           extra = "merge") %>%

  select(-temp) %>%

  mutate(Income = as.numeric(Income)) %>%

  mutate(

    Discount_Applied = case_when(

      Discount_Applied %in% c("yes", "YES", "Yes") ~ "Yes",

      Discount_Applied %in% c("no", "NO", "No") ~ "No",

      TRUE ~ Discount_Applied
    )
  )
```

```
## Warning: Expected 2 pieces. Additional pieces discarded in 790 rows [1, 2, 3, 4, 5, 6,
## 7, 8, 9, 10, 11, 12, 13, 14, 16, 17, 18, 19, 20, 21, ...].
```

```r
transaction_data_2 <- transaction_data_2 %>%

  mutate(

    Gender = case_when(

      Gender == "M" | Gender == "Male" ~ "Male",

      Gender == "F" | Gender == "Female" ~ "Female",

      TRUE ~ "Other"
    ),

    Payment_Method = case_when(

      Payment_Method == "Debit" | Payment_Method == "Debit card" ~ "Debit Card",

      Payment_Method == "Credit" | Payment_Method == "Credit card" ~ "Credit Card",

      TRUE ~ Payment_Method
    ),

    Category = case_when(

      Category == "Elects" | Category == "Electronic" ~ "Electronics",

      Category == "Clothes" | Category == "Cloth" ~ "Clothing",

      TRUE ~ Category
    )
  )
```

- Merge `transaction_data_1` and `transaction_data_2`: the unmatched rows in either input should not be included in the output. Name the output `transaction_data`.

```r
transaction_data <- transaction_data_1 %>%

                    inner_join(transaction_data_2, by = "Transaction_ID")
```

- Add a new variable, `Total_Amount`, to `transaction_data`, which represents the total cost of each transaction.

```r
transaction_data <- transaction_data %>%

                mutate(Total_Amount = Quantity * Product_Price + Shipping_Cost)
```

Use transaction_data for the remaining problems.

Question 2:

- Compute the average `Total_Amount` per transaction for each product `Category` and `Payment_Method` combination. Which payment method is associated with higher spending in each category?

```
avg_spending <- transaction_data %>%

                group_by(Category, Payment_Method) %>%

                summarise(Average_Total = mean(Total_Amount), .groups = "drop")


print(avg_spending)
```

```
## # A tibble: 12 x 3
##    Category    Payment_Method Average_Total
##    <chr>       <chr>                  <dbl>
##  1 Clothing    Credit Card             794.
##  2 Clothing    Debit Card              778.
##  3 Clothing    Other                   932.
##  4 Clothing    PayPal                  758.
##  5 Electronics Credit Card             786.
##  6 Electronics Debit Card              697.
##  7 Electronics Other                   776.
##  8 Electronics PayPal                  838.
##  9 Household   Credit Card             728.
## 10 Household   Debit Card              773.
## 11 Household   Other                  1022.
## 12 Household   PayPal                  941.
```

```
highest_spending <- avg_spending %>%

                    group_by(Category) %>%

                    slice_max(Average_Total, n = 1) %>%

                    ungroup()

print(highest_spending)
```

```
## # A tibble: 3 x 3
##   Category    Payment_Method Average_Total
##   <chr>       <chr>                  <dbl>
## 1 Clothing    Other                   932.
## 2 Electronics PayPal                  838.
## 3 Household   Other                  1022.
```

- Create age groups (`<25`, `25-40`, `41-60`, `>60`). Within each age group, calculate the percentage of customers who used `Credit Card` as the payment method. What does this say about payment preferences by age?

```r
transaction_data <- transaction_data %>%

                    mutate(Age_Group = case_when(

                    Age < 25 ~ "<25",

                    Age >= 25 & Age <= 40 ~ "25-40",

                    Age >= 41 & Age <= 60 ~ "41-60",

                    Age > 60 ~ ">60"))


credit_card_pct <- transaction_data %>%

  group_by(Age_Group) %>%

  summarise(

    Total_Customers = n(),

    Credit_Card_Users = sum(Payment_Method == "Credit Card"),

    Percentage_Credit_Card = (Credit_Card_Users / Total_Customers) * 100,

    .groups = "drop")


print(credit_card_pct)
```

```
## # A tibble: 4 x 4
##   Age_Group Total_Customers Credit_Card_Users Percentage_Credit_Card
##   <chr>               <int>             <int>                  <dbl>
## 1 25-40                 221               119                   53.8
## 2 41-60                 312               154                   49.4
## 3 <25                    96                42                   43.8
## 4 >60                   171                94                   55.0
```

- Split customers into `income` quartiles (below 25th percentile, between 25th and 50th percentiles, between 50th and 75th percentile, and above 75th percentile). Within each quartile, calculate the percentage of transactions with a discount applied. Interpret whether discounts tend to attract higher- or lower-income customers.

```r
transaction_data <- transaction_data %>%

  mutate(Income_Quartile = case_when(

    Income <= quantile(Income, 0.25) ~ "Q1",

    Income > quantile(Income, 0.25) & Income <= quantile(Income, 0.5) ~ "Q2",

    Income > quantile(Income, 0.5) & Income <= quantile(Income, 0.75) ~ "Q3",
```

```
    Income > quantile(Income, 0.75) ~ "Q4"))


discount_pct <- transaction_data %>%
  group_by(Income_Quartile) %>%
  summarise(
    Total_Transactions = n(),
    Discount_Transactions = sum(Discount_Applied == "Yes"),
    Percentage_Discount = (Discount_Transactions / Total_Transactions) * 100,
    .groups = "drop")

print(discount_pct)
```

```
## # A tibble: 4 x 4
##   Income_Quartile Total_Transactions Discount_Transactions Percentage_Discount
##   <chr>                        <int>                <int>               <dbl>
## 1 Q1                             222                   59                26.6
## 2 Q2                             201                   67                33.3
## 3 Q3                             219                   57                26.0
## 4 Q4                             158                   43                27.2
```

- Identify all customers who made purchases in **at least 2 different categories**. Among these multi-category shoppers, compute the average `Total_Amount` per transaction and average `rating`.

```
multi_category_customers <- transaction_data %>%
                            group_by(Customer_ID) %>%
                            summarise(
                            Num_Categories = n_distinct(Category),
                            .groups = "drop") %>%
                            filter(Num_Categories >= 2)

multi_category_transactions <- transaction_data %>%
                  filter(Customer_ID %in% multi_category_customers$Customer_ID)

multi_category_summary <- multi_category_transactions %>%
                          summarise(
```

```
                        Average_Total_Amount = mean(Total_Amount),

                        Average_Rating = mean(Customer_Rating, na.rm = TRUE))


print(multi_category_summary)
```

```
##   Average_Total_Amount Average_Rating
## 1              783.146       3.779174
```

- Determine how many unique customers made multiple transactions. Among customers who made more than one transaction, calculate the average number of transactions per customer.

```
customer_txn_counts <- transaction_data %>%

                       group_by(Customer_ID) %>%

                       summarise(

                       Num_Transactions = n(),

                       .groups = "drop")



multi_txn_customers <- customer_txn_counts %>%

                       filter(Num_Transactions > 1)


num_unique_multi_txn_customers <- nrow(multi_txn_customers)


avg_transactions_per_customer <- mean(multi_txn_customers$Num_Transactions)

print(num_unique_multi_txn_customers)
```

```
## [1] 242
```

```
print(avg_transactions_per_customer)
```

```
## [1] 2.921488
```

Question 3:

- Create a scatter plot of `Income` vs `Total_Amount`. Add marginal histograms on x and y axes, color points by `Discount_Applied`, and add a title and axis labels.

```r
gg <- ggplot(data = transaction_data) +
    geom_point(mapping = aes(x = Income, y = Total_Amount, color = Discount_Applied)) +
    labs(title = "Transaction Data", x = "Income", y = "Total Amount")
ggMarginal(gg, type = "histogram")
```

